Procedural vs. Object Thinking

Complete Notes, Code Solutions & Architectural Comparison

Topic Overview

Limitations of C (Procedural Approach)

- **Global Data Exposure:** Global data can be accessed or modified by *any function* with zero protection or access control.
- **Lack of Encapsulation:** Data and the functions operating on it live completely separate lives.
- **Scalability Bottlenecks:** As the codebase grows, tracking which functions manipulate which global variables becomes overwhelmingly messy.
- **Discipline-Dependent Integrity:** Data integrity relies entirely on programmer discipline rather than enforced language rules.

Why Object-Oriented Programming (OOP) Solves These Issues

- **Encapsulation:** Data fields and methods are bundled together inside a single class, preventing outside code from directly corrupting internal states.
- **Access Control:** Keywords like `private` and `public` strictly enforce who can interact with specific data.
- **Reusability & Real-World Modeling:** Classes model real-world entities naturally (e.g., an individual `Task` or a container `TaskList`).

C Programming Tasks

Task 1: Global Array Task List (C)

```c
// tasklist_cc.c
#include <stdio.h>
#include <string.h>

char tasks[5][100];
int taskCount = 0;

void addTask(const char *title) {
    if (taskCount < 5) {
        strcpy(tasks[taskCount], title);
        taskCount++;
    } else {
        printf("Task list is full!\n");
    }
}

void printTasks() {
```

```
    printf("\n--- Your Tasks ---\n");
    for (int i = 0; i < taskCount; i++) {
        printf("%d. %s\n", i + 1, tasks[i]);
    }
}

int main() {
    addTask("Buy groceries");
    addTask("Study C programming");
    addTask("Go for a walk");
    printTasks();
    return 0;
}
```

Task 2: Marking Tasks as Done via String Suffix (C)

```
// tasklist_cc_v2.c
#include <stdio.h>
#include <string.h>

char tasks[5][100];
int taskCount = 0;

void addTask(const char *title) {
    if (taskCount < 5) {
        strcpy(tasks[taskCount], title);
        taskCount++;
    }
}

void markTaskDone(int index) {
    if (index < 0 || index >= taskCount) {
        printf("Invalid task index!\n");
        return;
    }
    // Avoid marking twice
    if (strstr(tasks[index], "- DONE") == NULL) {
        strcat(tasks[index], " - DONE");
    }
}
```

Python OOP Solutions

Task 3 & 4: Task and TaskList Classes (Python)

```
class Task:
    def __init__(self, title: str):
        self.title = title
        self.isDone = False     # New tasks start pending

    def markDone(self):
        self.isDone = True
```

```
    def display(self):
        status = "DONE [OK]" if self.isDone else "PENDING"
        print(f"{self.title} [{status}]")

class TaskList:
    def __init__(self):
        self.tasks = []         # Private-ish storage for Task objects

    def addTask(self, title: str):
        self.tasks.append(Task(title))

    def markTaskDone(self, index: int):
        if 0 <= index < len(self.tasks):
            self.tasks[index].markDone()
        else:
            print("Invalid task index!")

    def showTasks(self):
        print("\n--- Your Tasks ---")
        for i, task in enumerate(self.tasks):
            print(f"{i + 1}. ", end="")
            task.display()

if __name__ == "__main__":
    my_list = TaskList()
    my_list.addTask("Buy groceries")
    my_list.addTask("Study C programming")
    my_list.markTaskDone(1)
    my_list.showTasks()
```

Comprehensive Architectural Comparison

#	Problem in the C Version	How OOP Solves It
1	**Unsafe global data** – `tasks[]` and `taskCount` are wide open to accidental corruption anywhere.	Data is encapsulated inside the object. Only authorized methods can modify internal state safely.
2	**Separated data & behavior** – Status relies on fragile string hacks (`" - DONE"`).	Data and behavior live together; status is a clean boolean property, eliminating double-marking bugs.
3	**Rigid scaling limits** – Hard-coded array sizes and manual index passing across functions.	Reusable, extensible classes allow unlimited lists and easy attribute additions (due dates, priorities).

Key Takeaway: In C, data is exposed and functions act externally, requiring strict discipline. In OOP, data and protective methods reside together within a class, allowing the object to guarantee its own state consistency.